	UNIVERSIDAD DEL MAGDALENA **VICERRECTORÍA ACADÉMICA** **DIRECCIÓN CURRICULAR Y DE DOCENCIA** **FORMATO PARA LA ELABORACIÓN DE MICRODISEÑO DE CURSO**	**Código: GA-DCD-F02** **Versión: 08** **Fecha: 06/06/2023**

PERIODO ACADÉMICO:	2026-3

1. IDENTIFICACIÓN DEL CURSO

1.1 Código	1.2 Nombre del curso	1.3 Tipo de curso		1.4 Pre-Requisito	1.5 Co-Requisit
	Programación y diseño orientado a objetos	Obligatorio	Teórico/Práctico	Fundamentos de programación	No aplica

1.6 N° Créditos	1.7 N° Semanas	1.8 HAD (Horas de Acompañamiento Directo)			1.9 HTI (Horas de Trabajo Independiente)	1.10 Tot HAD
			1.8.1 Horas presenciales Aula Clase	48		
			1.8.2 Horas presenciales Salida de Campo y/o Laboratorio	0		
			1.8.3 Horas Espacio Virtual	0		
3	12	48			96	1

1.11 Facultad/ Centro /Departamento	Facultad de Ingeniería	**1.12 Programa Académico**	
1.13 Área de Formación	Formación disciplinar	**1.14 Componente**	Fundamentos de computación

2. JUSTIFICACIÓN DEL CURSO DENTRO DEL ÁREAy/o COMPONENTE

El curso Programación y diseño orientado a objetos da continuidad a Fundamentos de programación dentro del componente de Fundamentos de computación del programa de In Ciencia de Datos. Parte de programas organizados mediante funciones y clases simples y lleva al estudiante hacia soluciones basadas en el diseño orientado a objetos, en las que la de procesamiento y análisis descriptivo de datos se representan mediante entidades, responsabilidades, relaciones e interacciones, se especifican con una notación estándar de m codifican en C#, como lenguaje principal, y en Python, para el análisis descriptivo de los datos.

El curso desarrolla el criterio de diseño necesario para construir programas mantenibles, consistentes y verificables, con persistencia básica de datos y pruebas unitarias, de integr funcionales. Esta base sostiene los cursos posteriores del componente, en particular Estructuras de Datos, donde las estructuras se representan y utilizan como objetos, y los curs programa que exigen construir software para el procesamiento, análisis y gestión de datos.

3. COMPETENCIAS A DESARROLLAR DENTRO DEL ÁREAy/o COMPONENTE

3.1 Competencias Genéricas

Capacidad para expresar y comprender ideas de manera clara y coherente en contextos académicos y profesionales.
Capacidad para analizar, evaluar alternativas y formular juicios fundamentados frente a situaciones problemáticas.
Capacidad para interactuar y colaborar de manera efectiva con otros en el desarrollo de actividades académicas y profesionales.
Capacidad para gestionar de manera responsable el propio proceso de aprendizaje.
Capacidad para actuar con compromiso, honestidad y respeto por las normas y buenas prácticas del trabajo académico y profesional.

3.2 Competencias Específicas

Capacidad para analizar necesidades de procesamiento y análisis descriptivo de datos e interpretarlas en términos de entidades, responsabilidades, reglas e interacciones entre ob
Capacidad para especificar el diseño de programas orientados a objetos mediante modelos estructurales y de interacción en una notación estándar de modelado, con base en prin patrones de diseño.
Capacidad para codificar programas orientados a objetos a partir de especificaciones de diseño, aplicando buenas prácticas de codificación, manejo de excepciones, reutilización y de datos.
Capacidad para verificar programas orientados a objetos mediante pruebas unitarias, de integración y funcionales que permitan localizar y corregir fallas.

4. RESULTADOS DE APRENDIZAJE DEL CURSO

Analiza especificaciones de funcionalidades derivadas de necesidades de procesamiento y análisis descriptivo de datos en términos de entidades, responsabilidades, reglas e inter desde un enfoque orientado a objetos.

Especifica clases, atributos, métodos, relaciones e interacciones que conformarán un programa de computador orientado al procesamiento y análisis descriptivo de datos, median de una notación estándar de modelado y con base en principios y patrones de diseño orientado a objetos.

Codifica programas de computador orientados al procesamiento y análisis descriptivo de datos en un lenguaje de programación orientado a objetos, con base en especificaciones buenas prácticas de codificación, manejo de excepciones, reutilización y persistencia de datos en archivos.

Verifica la ejecución de programas de computador orientados al procesamiento y análisis descriptivo de datos mediante pruebas unitarias, pruebas de integración y pruebas funci aplicadas a flujos principales, flujos alternativos y condiciones límite.

4.1 Evidencia 1:	4.2 Actividades de aprendizaje 1:	4.1 Estrategias pedagógicas 1:
Documento de análisis de funcionalidades que identifica, para una necesidad de procesamiento y análisis descriptivo de datos, las entidades del dominio, sus responsabilidades, las reglas y restricciones aplicables y las interacciones entre objetos en escenarios principales, alternativos y límite.	Preparación (trabajo independiente previo): Revisión de documentos, videos y ejercicios guiados sobre la identificación de entidades, responsabilidades, reglas e interacciones a partir de especificaciones de funcionalidades. Trabajo en clase (sesión sincrónica): Explicación breve de los conceptos y desarrollo de ejercicios en equipo para analizar especificaciones de funcionalidades de procesamiento y análisis descriptivo de datos, identificar las entidades del dominio y asignarles responsabilidades. Actividades de profundización (trabajo independiente posterior): Desarrollo de ejercicios adicionales para delimitar reglas, restricciones e interacciones en escenarios principales, alternativos y límite, revisando y ajustando los análisis elaborados en clase.	Estrategia: aula invertida con aprendizaje basado en problemas. Estudio de casos: análisis de especificaciones de funcionalidades de p y análisis descriptivo de datos para identificar las entidades del domi Análisis guiado de responsabilidades: asignación de responsabilidade colaboraciones a las entidades mediante tarjetas CRC. Análisis de escenarios: recorrido de escenarios principales, alternativ para delimitar reglas, restricciones e interacciones. Revisión y discusión en equipo: contraste colaborativo de los análisis para justificar las abstracciones elegidas.

Modelos de diseño elaborados en UML, con diagramas de clases y diagramas de secuencia, que especifican clases, atributos, métodos, relaciones e interacciones, y justifican los principios y patrones de diseño aplicados.	Preparación (trabajo independiente previo): Revisión de documentos, videos y ejercicios guiados sobre diagramas de clases y de secuencia en UML, principios de diseño orientado a objetos y patrones básicos de diseño. Trabajo en clase (sesión sincrónica): Explicación breve de los conceptos y desarrollo de ejercicios en equipo para construir diagramas de clases y de secuencia a partir de los análisis de funcionalidades, y para discutir la aplicación de principios y patrones de diseño. Actividades de profundización (trabajo independiente posterior): Desarrollo de ejercicios adicionales para completar y refinar los modelos de diseño, verificando su consistencia interna y su correspondencia con las funcionalidades analizadas.	Estrategia: aula invertida con aprendizaje basado en problemas. Modelado guiado: construcción paso a paso de diagramas de clases y a partir del análisis de funcionalidades. Análisis de modelos: revisión de diagramas existentes para identificar inconsistencias, dependencias innecesarias o responsabilidades mal a Estudio de patrones: análisis de problemas de diseño recurrentes y d básicos que los resuelven. Revisión y mejora de diseños: discusión y ajuste de los modelos con b cohesión, acoplamiento y reutilización.
Programas orientados a objetos desarrollados en C# como lenguaje principal, con un componente en Python para el análisis descriptivo de los datos persistidos, que implementan los modelos de diseño aplicando buenas prácticas de codificación, manejo de excepciones, mecanismos de reutilización y persistencia de datos en archivos.	Preparación (trabajo independiente previo): Revisión de documentos, videos y ejercicios guiados sobre la implementación en C# de clases, relaciones, herencia, polimorfismo, interfaces, genéricos, manejo de excepciones y persistencia de datos en archivos, y sobre sus equivalentes en Python. Trabajo en clase (sesión sincrónica): Explicación breve de los conceptos y práctica individual supervisada: cada estudiante implementa en C# los modelos de diseño elaborados y los ejecuta antes de la revisión en equipo. En segmentos breves se contrasta la misma solución en Python. Durante la sesión se analizan fragmentos de código para verificar su correspondencia con el diseño y mejorar su organización. Actividades de profundización (trabajo independiente posterior): Desarrollo de ejercicios adicionales para completar, reorganizar o extender programas orientados a objetos, incorporando reutilización, manejo de excepciones y carga y almacenamiento de datos en archivos.	Estrategia: práctica guiada con andamiaje decreciente y aprendizaje b proyectos. Programación guiada: construcción paso a paso de programas orienta a partir de los modelos de diseño, con acompañamiento del profesor. Lectura y análisis de código: revisión de programas para verificar su correspondencia con el diseño e identificar mejoras en su organizació Refactorización de programas: reorganización de código existente pa encapsulamiento, cohesión y reutilización. Programación colaborativa: desarrollo de programas en pequeños eq discutir decisiones de implementación. Revisión de código entre pares: análisis y comentarios sobre los progr elaborados por otros estudiantes para mejorar su calidad.
Conjuntos de pruebas unitarias, de integración y funcionales ejecutadas sobre los programas desarrollados, con resultados documentados para flujos principales, flujos alternativos y condiciones límite, y registro de las fallas corregidas.	Preparación (trabajo independiente previo): Revisión de documentos, videos y ejercicios guiados sobre pruebas unitarias, pruebas de integración y pruebas funcionales, y sobre el uso de marcos de pruebas automatizadas en C# y Python. Trabajo en clase (sesión sincrónica): Explicación breve de los conceptos y desarrollo de ejercicios en equipo para diseñar casos de prueba, automatizar pruebas unitarias y de integración, ejecutar pruebas funcionales y analizar los resultados obtenidos. Actividades de profundización (trabajo independiente posterior): Desarrollo de ejercicios adicionales para ampliar los conjuntos de pruebas sobre flujos principales, alternativos y condiciones límite, documentando los resultados y corrigiendo las fallas detectadas.	Estrategia: aprendizaje basado en proyectos con verificación sistemát Diseño guiado de casos de prueba: definición de casos de prueba a pa responsabilidades de las clases, los flujos y las condiciones límite. Automatización de pruebas: construcción y ejecución de pruebas unit integración con marcos de pruebas de C# y Python. Discusión de resultados de prueba: análisis colectivo de las fallas dete identificar su origen. Revisión y corrección de programas: ajuste de los programas a partir de las pruebas.

5. PROGRAMACIÓN DEL CURSO

Semana	Unidad Temática	Contenido de Aprendizaje	HAD		
			Horas presenciales Aula Clase	Horas presenciales Salida de Campo y/o Laboratorio	Horas Espacio Virtual
1	Iniciación en C# y listas	Introducción al curso Variables en C# Python vs C# Condicionales Ciclos Funciones Listas Arrays LINQ Expresiones lambda	4	0	0

2	Clases, objetos y abstracción	Definición de clase Definición de objeto Creación de objetos Atributos de instancia Atributos calculados Definición de abstracción Tarjetas CRC Constructores de objeto Taller CRC	4	0	0
3	Encapsulamiento	Encapsulamiento y control de acceso Validaciones de acceso en atributos Validaciones en procedimientos Constructores públicos o privados Taller de encapsulamiento	4	0	0
4	Encapsulamiento	Socialización del taller Generalidades al construir clases Trucos para detectar dominios Trucos para detectar comportamientos y atributos Primer examen	4	0	0
5	Herencia y Relaciones entre objetos	Herencia y especialización Relaciones entre objetos Composición Variables por valor y por referencia	4	0	0
6	Polimorfismo y Excepciones	Comportamientos especializados de métodos por entidades Polimorfismo Ampliación en el manejo de excepciones Uso de throw con excepciones personalizadas Uso de try-catch Seguimiento evaluativo	4	0	0
7	Interfaces, clases abstractas y UML	Definición de UML Usos de UML Diagramas de clases Cardinalidad Diagramas de secuencia Definición de interfaces Clases abstractas Documentación	4	0	0
8	Principios y patrones de diseño	Cohesión Acoplamiento Separación de responsabilidades Principio de responsabilidad única Principio abierto-cerrado Principio de inversión de dependencias Patrón Factory Method Patrón Estrategia	4	0	0
9	Persistencia básica de datos, serialización y Data Transfer Object (DTO)	Definición de serialización Deserialización Formatos XML y JSON Definición de Data Transfer Object (DTO) Representación persistente de objetos Lectura y escritura de archivos Segundo examen	4	0	0
10	Pruebas unitarias y de integración	Pruebas unitarias Pruebas de integración Documentación en código Casos de prueba Uso de Fact y Theory Framework xUnit Taller	4	0	0
11	Análisis descriptivo y verificación funcional	Consultas sobre repositorios con LINQ Filtrado y conteo Promedio y agrupación Pruebas funcionales de flujos principales Pruebas de flujos alternativos y condiciones límite Taller	4	0	0
12	Evaluación	Entrega técnica del trabajo final Retroalimentación y preparación de la sustentación oral Sustentación oral individual	4	0	0
		Total Horas HAD+HTI:	48	0	0
		CRÉDITOS ACADÉMICOS	**3**		

6. ESCENARIO PRÁCTICAS DE CAMPO (Diligenciar solo si aplica)

Departamento	Ciudad, Municipio, Corregimiento y/o vereda	Lugar de desarrollo de la práctica de campo	Propósito de la práctica de campo	Resultado de Aprendizaje	Recursos, M impler
N/A	N/A	N/A	N/A	N/A	N/A

7. ESCENARIO LABORATORIO (Diligenciar solo si aplica)

Lugar de desarrollo de la práctica de laboratorio	Propósito de la práctica de laboratorio	Resultado de Aprendizaje	Recursos, Materiales y/o implementos

Sala de computadores (aula habitual del curso)	Modelar, codificar y probar programas orientados a objetos propios con acompañamiento del profesor, y avanzar en el proyecto	Todos los definidos para el curso	Sala de computadores con Visual Studio Code (C# De extensión de Python), .NET SDK en versión LTS vigent o superior, PlantUML o diagrams.net, xUnit y pytest de cobertura, y Git

8. VALORACIÓN DE LOS RESULTADOS DE APRENDIZAJE (mínimo debe establecer tres (3) actividades evaluativas)

N° Actividad Evaluativa	Semana	Actividad de Evaluación del Aprendizaje	Objetivo	Descripción	Mecanismo, Criterio y/o Rúbrica	Valoración (0-500)
1	2	Prueba corta individual	Verificar la aplicación de los conceptos y procedimientos trabajados en las sesiones de las semanas 1 y 2	Prueba práctica individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# las clases y relaciones correspondientes que cumplan con la solución del enunciado.	Mecanismo: Prueba práctica individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 1.1 Implementación y funcionamiento 1.2 Organización del código 2.1 Tarjetas CRC 2.2 Abstracción y atributos 2.3 Métodos y responsabilidades 2.4 Pruebas de la solución Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	20
2	3	Prueba corta individual	Verificar la aplicación de los conceptos y procedimientos trabajados en las sesiones de la semana 3	Prueba práctica individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# las clases y relaciones correspondientes que cumplan con la solución del enunciado.	Mecanismo: Prueba práctica individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 2.1 Abstracción 2.2 Comportamientos de la clase 3.1 Diseño y encapsulamiento 3.2 Validaciones 3.3 Casos de prueba Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	20
3	4	Examen individual	Evaluar la capacidad del estudiante para analizar problemas y aplicar los conceptos y técnicas trabajados en las semanas 1 a 4 del curso	Prueba práctica individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# las clases y relaciones correspondientes que cumplan con la solución del enunciado.	Mecanismo: Prueba práctica individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 1.1 Condicionales, ciclos y funciones en C# 1.2 Listas, arrays y LINQ 2.1 Creación de objetos y constructores 2.2 Atributos de instancia y calculados 2.3 Abstracción con tarjetas CRC 3.1 Encapsulamiento y control de acceso 3.2 Validaciones en atributos y procedimientos 3.3 Constructores públicos o privados 4.1 Identificación de dominio, comportamientos y atributos Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	115
4	7	Prueba corta individual	Verificar la aplicación de los conceptos y procedimientos trabajados en las sesiones de las semanas 4 a 6	Prueba práctica individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# las clases y relaciones correspondientes que cumplan con la solución del enunciado.	Mecanismo: Prueba práctica individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 5.1 Herencia 5.2 Relación entre objetos 6.1 Polimorfismo 6.2 Manejo de excepciones Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	20

5	8	Prueba corta individual	Verificar la aplicación de los conceptos y procedimientos trabajados en las sesiones de las semanas 7 y 8	Prueba práctica individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# las clases y relaciones correspondientes que cumplan con la solución del enunciado.	Mecanismo: Prueba práctica individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 7.1 Interfaces y clases abstractas 7.2 UML Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	20
6	9	Examen individual	Evaluar la capacidad del estudiante para analizar problemas y aplicar los conceptos y técnicas trabajados en las semanas 1 a 9 del curso	Prueba práctica individual en sala de computadores: a partir de una especificación y un diagrama de clases parcial, el estudiante completa el diagrama e implementa en C# las clases y relaciones correspondientes. La solución debe superar los casos de prueba ocultos del docente.	Mecanismo: Prueba práctica individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba ocultos y se revisa con la rúbrica. Criterio: Se valoran los indicadores: 2.1 Abstracción 3.1 Encapsulamiento y validaciones 4.1 Identificación de dominio, comportamientos y atributos 5.1 Herencia y especialización 5.2 Relaciones entre objetos y composición 6.1 Polimorfismo 7.1 Interfaces y clases abstractas 8.2 Patrón Factory Method o Estrategia Rúbrica: Cada indicador se valora en tres niveles: desempeño insuficiente, desempeño mínimo de aprobación y desempeño máximo alcanzable, según las descripciones del marco curricular del curso.	115
7	10	Taller práctico individual	Verificar la aplicación de los conceptos y procedimientos de serialización, deserialización y DTO trabajados en la semana 9	Taller práctico individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# la serialización y deserialización de objetos en JSON o XML utilizando DTO, que cumplan con la solución del enunciado.	Mecanismo: Taller práctico individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 9.1 Serialización y deserialización en JSON o XML 9.2 Data Transfer Object (DTO) Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	20
8	11	Taller práctico individual	Verificar la aplicación de los conceptos y procedimientos de serialización, deserialización, DTO y pruebas unitarias, de integración y funcionales trabajados en las semanas 9 a 11	Taller práctico individual en sala de computadores: a partir de una especificación (dominio), el estudiante implementa en C# la serialización y deserialización de objetos utilizando DTO y escribe pruebas unitarias, de integración y funcionales con xUnit que verifiquen su funcionamiento en flujos principales, alternativos y condiciones límite.	Mecanismo: Taller práctico individual de 2 horas en sala de computadores. La solución se ejecuta contra casos de prueba realizados por el docente y se validan las reglas enseñadas al estudiante. Criterio: Se valoran los indicadores: 9.1 Serialización y deserialización en JSON o XML 9.2 Data Transfer Object (DTO) 10.1 Pruebas unitarias con xUnit (Fact y Theory) 10.2 Pruebas de integración 11.1 Pruebas funcionales de flujos principales 11.2 Pruebas de flujos alternativos y condiciones límite Rúbrica: Cada ítem tiene un puntaje máximo y se contrasta con lo entregado por el estudiante.	20

9	12	Examen oral individual sobre trabajo de fin de curso	Valorar de manera integral la capacidad del estudiante para explicar, justificar y demostrar el funcionamiento de la solución desarrollada como trabajo final del curso	Evaluación oral individual basada en el proyecto final entregado. Cada estudiante responde preguntas formuladas por el profesor, seleccionadas de manera aleatoria conforme a lo entregado en el proyecto final (análisis de funcionalidades, modelos de diseño, implementación del programa y pruebas). La evaluación se centra en la capacidad del estudiante para explicar, justificar y analizar su contribución a la solución, y comprueba la autoría de los productos entregados. Requisito para la sustentación: el proyecto final debe cumplir con lo requerido en el enunciado.	Mecanismo: Evaluación oral individual de 8 a 10 minutos con preguntas seleccionadas de manera aleatoria sobre lo entregado en el proyecto final. Requisito: El proyecto final debe cumplir con lo requerido para tener derecho a la sustentación. Criterio: En cada pregunta se valora el dominio del estudiante sobre el aspecto del proyecto seleccionado. Rúbrica: Cada pregunta se valora en tres niveles: desempeño insuficiente, desempeño mínimo de aprobación y desempeño máximo alcanzable, según las descripciones del marco curricular del curso.	150

Condición de aprobación: todos los resultados de aprendizaje deben contar con evidencia calificable, registrada en la entrega técnica y en la sustentación oral. Un desempeño alto resultados puede compensar debilidades acotadas en otros, pero ningún estudiante aprueba el curso cuando alguno de los resultados carece por completo de evidencia de logro.

9. RECURSOS EDUCATIVOS Y HERRAMIENTAS TIC

N°	Nombre	Justificación	Contenido de Aprendizaje
1	Curso en campus virtual	Espacio virtual de interacción para compartir recursos y actividades de aprendizaje del curso	Todos los contenidos del curso
2	Visual Studio Code con .NET SDK y Python	Entorno para escribir, ejecutar y depurar programas en C# y Python	Codificación, persistencia y pruebas
3	PlantUML o diagrams.net	Elaboración de diagramas de clases y de secuencia en UML	Modelado orientado a objetos
4	xUnit y pytest	Automatización de pruebas unitarias y de integración, con medición de cobertura	Verificación de programas orientados a objetos
5	Git y GitHub	Control de versiones del trabajo final y trazabilidad de los aportes individuales	Trabajo final del curso

10. REFERENCIAS BIBLIOGRÁFICAS

D. West, Object Thinking, 1st ed. Redmond, WA, USA: Microsoft Press, 2004.

M. Weisfeld, The Object-Oriented Thought Process, 5th ed. Boston, MA, USA: Addison-Wesley, 2019.

E. Gamma, R. Helm, R. Johnson y J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994.

E. Freeman y E. Robson, Head First Design Patterns, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

C. Larman, Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 200

R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

V. Khorikov, Unit Testing Principles, Practices, and Patterns. Shelter Island, NY, USA: Manning Publications, 2020.

B. Okken, Python Testing with pytest, 2nd ed. Raleigh, NC, USA: The Pragmatic Bookshelf, 2022.

Profesor(a) curso	Docente coordinador área y/o componente	Director(a) Programa Académico

o

tal Horas
+ HTI

44

igeniería en
as necesidades
nodelado y se

ración y
os del

ojetos.
ncipios y

y persistencia

acciones,

nte diagramas

de diseño,

onales

rocesamiento
nio.

s y

os y límite

elaborados

de secuencia

.

ısignadas.

e los patrones

ıase en

ɔasado en

ados a objetos

.

ɔn.

ra mejorar

uipos para

ramas

tica.

artir de las

tarias y de

ectadas para

de la evidencia

HTI
Trabajo Independiente
8

8
8
8
8
8
8
8
8
8
8
8
8
96
lateriales y/o mentos
s

v Kit y
te, Python 3.12
con medición

Porcentaje de Valoración
4
4
23
4

4
23
4
4

30

en algunos

4.